P3154R1
Deprecating signed character types in iostreams

Published Proposal, 2024-05-20

This version:
http://wg21.link/P3154R1.html

Author:
Elias Kosunen

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes deprecating overloads under iostreams, that take some variant of `signed char` or `unsigned char`, and treat these as characters, rather than integers. The behavior of these overloads is unexpected, especially when using the aliases `int8_t` or `uint8_t`.

Table of Contents

4.1 Modify [istream.general] p1

4.2 Modify [istream.extractors], around p7 to p12

4.3 Modify [ostream.general] p1

4.4 Modify [ostream.inserters.character]

4.5 Add a new subclause in Annex D after [depr.atomics]

4.6 Add a new subclause in Annex D after the above ([depr.istream.extractors])

References

Informative References

§ 1. Changelog

§ 1.1. Changes since R0

- Add a study on possible impact
- Update the code example in Motivation
- Add note about `char8_t` in C
- Add note about [P0487R1]

§ 2. Motivation

```
#include <iostream>
#include <format>

int main() {
    // Prints:
    std::cout
        << static_cast<         char>(48) << '\n'  // 0
        << static_cast<  signed char>(48) << '\n'  // 0 (Proposing depreca
        << static_cast<unsigned char>(48) << '\n'  // 0 (Proposing depreca
        << static_cast<       int8_t>(48) << '\n'  // 0 (Proposing depreca
        << static_cast<      uint8_t>(48) << '\n'  // 0 (Proposing depreca
        << static_cast<        short>(48) << '\n'  // 48

        << std::format("{}\n", static_cast<         char>(48))  // 0
        << std::format("{}\n", static_cast<  signed char>(48))  // 48
        << std::format("{}\n", static_cast<unsigned char>(48))  // 48
```

```
        << std::format("{}\n", static_cast<       int8_t>(48))  // 48
        << std::format("{}\n", static_cast<      uint8_t>(48))  // 48
        << std::format("{}\n", static_cast<        short>(48)); // 48
}
```

There are overloads for `operator<<` for `basic_ostream`, that take an `(un)signed char`, and a `const (un)signed char*`. In addition, there are overloads for `operator>>` for `basic_istream`, that take an `(un)signed char&` and an `(un)signed char (&)[N]`. These overloads are specified to have equivalent behavior to the non-signedness qualified overloads: [istream.extractors] [ostream.inserters.character].

This is surprising. Per [basic.fundamental] p1 and p2:

> There are five *standard signed integer types*: `"signed char"`, `"short int"`, `"int"`, `"long int"`, and `"long long int"`... There may also be implementation-defined *extended signed integer types*. The standard and extended signed integer types are collectively called *signed integer types*.
>
> For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: `"unsigned char"`, `"unsigned short int"`, `"unsigned int"`, `"unsigned long int"`, and `"unsigned long long int"`... Likewise, for each of the extended signed integer types, there exists a corresponding *extended unsigned integer types*. The standard and extended unsigned integer types are collectively called *unsigned integer types*.

Thus, `signed char` and `unsigned char` should be treated as integers, not as characters. This is highlighted by the fact, that `int8_t` and `uint8_t` are specified to be aliases to (un)signed integer types, which are in practice going to be `signed char` and `unsigned char`.

> NOTE: The Solaris implementation is different, and defines `int8_t` to be `char` by default. This is not conformant.

`signed char` and `unsigned char` are not character types. Per [basic.fundamental] p11, since [P2314R4]:

> The types `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t` are collectively called *character types*.

`signed char` and `unsigned char` are included in the set of *ordinary character types* and *narrow character types* ([basic.fundamental] p7), but these definitions are used for specifying alignment, padding, and *indeterminate values* ([basic.indet]), and are arguably not related to characters in the sense of pieces of text.

`std::format` has already taken a step in the right direction here, by treating `signed char` and `unsigned char` as integers. It's specified to not give special treatment to these types, but to use the standard definitions of (un)signed integer type to determine whether a type is to be treated as an integer when formatting.

This paper proposes that these overloads in iostreams should be deprecated.

§ 3. Impact

It's difficult to find examples where this is the sought-after behavior, and would become deprecated with this change. These snippets aren't easily greppable.

It's easy to find counter-examples, however, where workarounds have to be employed to insert or extract `signed char`s or `unsigned char`s as integers. Some of them can be found with isocpp.org codesearch by searching for `<< static_cast<int>` or `<< (int)`, although false positives there are very prevalent.

```
/* ... */ << static_cast<int>(my_schar);
```

These overloads have existed since C++98. The signature of `operator>>` for `basic_istream` was updated for C++20 in [P0487R1], where these functions were changed to take `T (&)[N]` instead of `T*`, for safety reasons. No other changes to these overloads have been made in standard C++.

```
// Changes in P0487, applied to C++20

template<class charT, class traits, size_t N>
  basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, un
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, si
```

It should be noted, that the C standard has defined `char8_t` to be an alias (typedef) to `unsigned char`. In C++, `char8_t` is a distinct type with an underlying type of `unsigned char`.

§ 3.1. Impact study

To gauge the potential impact of this deprecation, the author tried building open source C++ code bases, using a patched version of libc++. Below are the instances where the overloads proposed for deprecation were used in these builds.

For reference, the author built tensorflow-lite and Tenzir using a custom version of libc++ where these overloads were marked as `= delete`d. These code bases number ~1½ MLoC in total, with a large number of dependencies, are reasonably modern, and use iostreams.

§ 3.1.1. Abseil

The Abseil logging library seems to treat `signed char` and `unsigned char` as character types. This is likely because the syntax used by the library is very similar to that used by iostreams:

```
signed char my_schar = 65;
LOG(ERROR) << my_schar;
// Will output:
// E0520 13:49:47.968463  123694 absl_log.cpp:8] A
// where the message itself is the 'A' here -----^
```

Internally in the library, this is achieved with this overload set:

```
// Abseil, version 20230802.1:
// absl/log/internal/check_op.cc

void MakeCheckOpValueString(std::ostream& os, const char v) {
  if (v >= 32 && v <= 128) {
    os << "'" << v << "'";
  } else {
    os << "char value " << int{v};
  }
}

void MakeCheckOpValueString(std::ostream& os, const signed char v) {
  if (v >= 32 && v <= 128) {
```

```
    os << "'" << v << "'";
  } else {
    os << "signed char value " << int{v};
  }
}

void MakeCheckOpValueString(std::ostream& os, const unsigned char v) {
  if (v >= 32 && v <= 128) {
    os << "'" << v << "'";
  } else {
    os << "unsigned char value " << int{v};
  }
}
```

where `signed char` and `unsigned char` are explicitly and intentionally treated similarly to `char`, and are passed to an underlying `std::ostream`. Notably, the values between `32` and `128` are really treated as character values, as they are printed with `'single quotes around them'`, and are cast to integers otherwise.

§ 3.1.2. FlatBuffers

In the implementation of `flatc` (the FlatBuffers schema compiler), there's the following function template:

```
// Flatbuffers, version 23.5.26:
// src/annotated_binary_text_gen.cpp

template<typename T> std::string ToString(T value) {
  if (std::is_floating_point<T>::value) {
    std::stringstream ss;
    ss << value;
    return ss.value();
  } else {
    return std::to_string(value);
  }
}
```

where the proposed-for-deprecation overload of `operator<<` is instantiated, if `T` is `signed char` or `unsigned char`. The overloads are never actually called, but because the above code is using `if` instead of `if constexpr`, the compiler warns about usage, anyway.

The current behavior when using `signed char` or `unsigned char` is to use `std::to_string`, which formats the value as an integer, as the overload `std::to_string(int)` is picked in overload resolution.

§ 3.1.3. simdjson

The following piece of code is present in the implementation of simdjson:

```
// simdjson, version 3.9.1:
// include/simdjson/dom/document-inl.h

inline bool document::dump_raw_tape(std::ostream &os) const noexcept {
  uint32_t string_length;
  size_t tape_idx = 0;
  uint64_t tape_val = tape[tape_idx];
  uint8_t type = uint8_t(tape_val >> 56);
  os << tape_idx << " : " << type;
  // ...
  os << tape_idx << " : " << type << "\t// pointing to " <<
  // ...
  if (type == 'r')
  // ...
  switch (type) {
  case '"':
  // ...
  case 'l':
  // ...
  }
}
```

This member function is apparently intended to be used for debugging. The `tape` referenced is a library-internal representation of a parsed JSON document.

Above, `type` has the type of `uint8_t`, but is clearly treated as a character type. Its value is compared to character literals, and thus, when written to a `std::ostream`, is intended to be formatted as a character. The proposed deprecation would break this.

§ 3.1.4. yaml-cpp

In yaml-cpp, the following piece of code can be found, where the `operator<<` overload is called with `signed char` and `unsigned char`:

```
// yaml-cpp, version 0.8.0:
// include/yaml-cpp/node/convert.h

// Used with T=signed char and T=unsigned char
template <typename T>
typename std::enable_if<!std::is_floating_point<T>::value, void>::type
inner_encode(const T& rhs, std::stringstream& stream){
  stream << rhs;
}
```

This function template is instantiated and called when writing to an existing YAML document:

```
signed char my_schar = 65;
unsigned char my_uchar = 65;
auto node = YAML::Load("{schar: 0, uchar: 0}");
node["schar"] = my_schar;
node["uchar"] = my_uchar;
std::cout << node;
// Outputs: {schar: A, uchar: A}
```

It's unclear whether treating `signed char` as a character type here is the desired behavior, or simply an oversight caused by the usage of `std::stringstream`. Elsewhere in the library, `signed char` is treated unambiguously as an integer, whereas `unsigned char` is treated as a character:

```
signed char my_schar = 65;
unsigned char my_uchar = 65;
YAML::Emitter out;
out << YAML::BeginMap
    << YAML::Key << "schar"
    << YAML::Value << my_schar
    << YAML::Key << "uchar"
    << YAML::Value << my_uchar
    << YAML::EndMap;
std::cout << out.c_str();
```

```
// Outputs:
// schar: 65
// uchar: A
```

There are two long-standing issues against yaml-cpp to inquire about this inconsistency, without a resolution before the mailing deadline.

§ 3.1.5. Conclusion

Only four instances of use were found during this study, which is not a lot. Notably, only uses of `operator<<` taking a `signed char` or `unsigned char` were found. No uses of the array-version of `operator<<` or any of the `operator>>` overloads were identified.

In these four cases:

- One (probably) contained a bug, which could have been identified with the deprecation proposed here § 3.1.4 yaml-cpp
- One was essentially a false-positive, where the deprecated overloads were never called, only instantiated § 3.1.2 FlatBuffers
- Two were cases were the deprecated behavior was the desired one § 3.1.1 Abseil § 3.1.3 simdjson

So, the use in the wild for these overloads seems to be quite limited. In some cases, the current behavior is asked for, but it's difficult to ascertain whether the developers writing that code initially got tripped up by this behavior. With this deprecation, at least a single possible bug was identified, and it's possible even more could be found, once developers are forced to check their usages as their compilers start warning them. If anything, forcing users to cast to `char`/`int`/`unsigned` could be argued to be an increase in readability, in favor of relying on the current behavior with `signed char` and `unsigned char`.

§ 4. Wording

This wording is relative to [N4971].

§ 4.1. Modify [istream.general] p1

```
// ...

// [istream.extractors], character extraction templates
template<class charT, class traits>
  basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sig

template<class charT, class traits, size_t N>
  basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sig
```

§ 4.2. Modify [istream.extractors], around p7 to p12

```
template<class charT, class traits, size_t N>
  basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sig
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. After a sentry object is constructed, `operator>>` extracts characters and stores them into `s`. If `width()` is greater than zero, `n` is `min(size_t(width()), N)`. Otherwise `n` is `N`. `n` is the maximum number of characters stored.

Characters are extracted and stored until any of the following occurs:

- `n-1` characters are stored;
- end of file occurs on the input sequence;
- letting `ct` be `use_facet<ctype<charT>>(in.getloc())`, `ct.is(ct.space, c)` is `true`.

`operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.

If the function extracted no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

```
template<class charT, class traits>
  basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sig
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. A character is extracted from `in`, if one is available, and stored in `c`. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

§ 4.3. Modify [ostream.general] p1

```
// ...

// [ostream.inserters.character], character inserters
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, cha
```

```
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, sig
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, uns

template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, wch
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, cha
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, cha
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, cha
template<class traits>
  basic_ostream<wchar_t, traits>&
    operator<<(basic_ostream<wchar_t, traits>&, char8_t) = delete;
template<class traits>
  basic_ostream<wchar_t, traits>&
    operator<<(basic_ostream<wchar_t, traits>&, char16_t) = delete;
template<class traits>
  basic_ostream<wchar_t, traits>&
    operator<<(basic_ostream<wchar_t, traits>&, char32_t) = delete;

template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, con

template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, con
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, con

template<class traits>
  basic_ostream<char, traits>&
    operator<<(basic_ostream<char, traits>&, const wchar_t*) = delete;
template<class traits>
  basic_ostream<char, traits>&
    operator<<(basic_ostream<char, traits>&, const char8_t*) = delete;
template<class traits>
```

```
  basic_ostream<char, traits>&
    operator<<(basic_ostream<char, traits>&, const char16_t*) = delete;
template<class traits>
  basic_ostream<char, traits>&
    operator<<(basic_ostream<char, traits>&, const char32_t*) = delete;
template<class traits>
  basic_ostream<wchar_t, traits>&
    operator<<(basic_ostream<wchar_t, traits>&, const char8_t*) = delete;
template<class traits>
  basic_ostream<wchar_t, traits>&
    operator<<(basic_ostream<wchar_t, traits>&, const char16_t*) = delete;
template<class traits>
  basic_ostream<wchar_t, traits>&
    operator<<(basic_ostream<wchar_t, traits>&, const char32_t*) = delete;

// ...
```

§ 4.4. Modify [ostream.inserters.character]

```
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
// specialization
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, cha
// signed and unsigned
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, sig
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, uns
```

Effects: Behaves as a formatted output function of `out`. Constructs a character sequence `seq`. If `c` has type `char` and the character container type of the stream is not `char`, then `seq` consists of

`out.widen(c)`; otherwise `seq` consists of `c`. Determines padding for `seq` as described in [ostream.formatted.reqmts]. Inserts `seq` into `out`. Calls `os.width(0)`.

Returns: `out`.

```
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, c
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, co
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, co
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, co
```

Preconditions: `s` is not a null pointer.

Effects: Behaves like a formatted inserter (as described in [ostream.formatted.reqmts]) of `out`. Creates a character sequence `seq` of `n` characters starting at `s`, each widened using `out.widen()` ([basic.ios.members]), where `n` is the number that would be computed as if by:

- `traits::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const charT*`, and also for the overload where the first argument is of type `basic_ostream<char, traits>&` and the second is of type `const char*`,
- `char_traits<char>::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const char*`,
- ~~`traits::length(reinterpret_cast<const char*>(s))` for the other two overloads.~~

Determines padding for `seq` as described in [ostream.formatted.reqmts]. Inserts `seq` into `out`. Calls `width(0)`.

Returns: `out`.

§ 4.5. Add a new subclause in Annex D after [depr.atomics]

Deprecated `signed char` and `unsigned char` extraction [depr.istream.extractors]

The following function overloads are declared in addition to those specified in [istream.extractors]:

```
template<class traits>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
template<class traits>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. A character is extracted from `in`, if one is available, and stored in `c`. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

```
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
template<class traits, size_t N>
  basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. After a sentry object is constructed, `operator>>` extracts characters and stores them into `s`. If `width()` is greater than zero, `n` is `min(size_t(width()), N)`. Otherwise `n` is `N`. `n` is the maximum number of characters stored.

Characters are extracted and stored until any of the following occurs:

- `n-1` characters are stored;
- end of file occurs on the input sequence;
- letting `ct` be `use_facet<ctype<charT>>(in.getloc())`, `ct.is(ct.space, c)` is `true`.

`operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.

If the function extracted no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

§ 4.6. Add a new subclause in Annex D after the above ([depr.istream.extractors])

Deprecated `signed char` and `unsigned char` insertion [depr.ostream.inserters]

The following function overloads are declared in addition to those specified in [ostream.inserters]:

```
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out
```

Effects: Equivalent to: `return out << static_cast<char>(c);`.

```
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out
template<class traits>
  basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out
```

Effects: Equivalent to: `return out << reinterpret_cast<const char*>(s);`.

§ References

§ Informative References

[N4971]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 18 December 2023. URL: https://wg21.link/n4971

[P0487R1]

Zhihao Yuan. *Fixing operator>>(basic_istream&, CharT*) (LWG 2499)*. 23 August 2018. URL: https://wg21.link/p0487r1

[P2314R4]

Jens Maurer. *Character sets and encodings*. 15 October 2021. URL: https://wg21.link/p2314r4